Alocação dinâmica de vetores e matrizes

Prof. Thiago Felski Pereira, MSc.

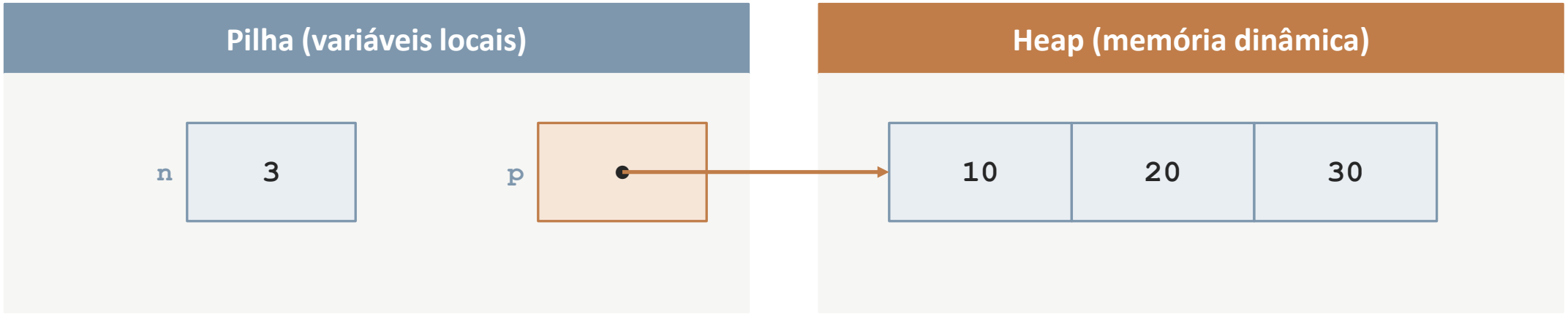
O problema: o tamanho só se sabe na execução

```
int n;
cin >> n;
int v[n];      // não é C++ padrão
```

- Em C++ padrão, o tamanho de um vetor deve ser conhecido **ao compilar**. Alguns compiladores aceitam `v[n]`, outros recusam.
- O jeito antigo: declarar `int v[1000]` e torcer. Ou falta espaço, ou sobra desperdício.
- A solução: pedir memória ao sistema **durante a execução**.

UNIVALI

Onde ficam as variáveis?

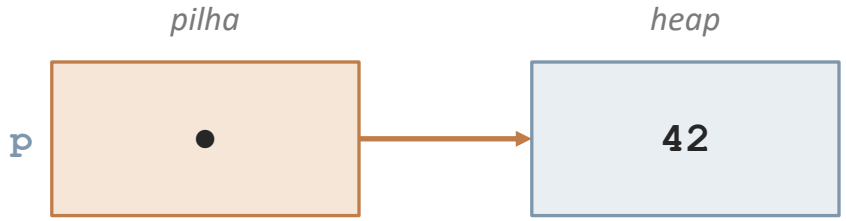


- **Pilha:** variáveis locais e parâmetros. Nascem ao entrar na função e **morrem ao sair**.
- **Heap:** memória que o programa pede com `new`. Vive **até o delete**, mesmo depois que a função termina.
- O ponteiro (na pilha) guarda o endereço da memória dinâmica (no heap).

UNIVALI

new: criando uma variável dinâmica

```
int *p = new int;
*p = 42;
cout << *p;
delete p;
```



- `new int` reserva espaço para um `int` no heap e **devolve o endereço**.
- A variável dinâmica não tem nome: só chegamos nela pelo ponteiro.
- `delete p` **devolve** a memória ao sistema.

UNIVALI

Toda alocação precisa de uma liberação

```
int *p = new int;
*p = 42;
p = new int;      // o primeiro int ficou perdido!
delete p;
```

- **Vazamento de memória:** a área continua reservada, mas ninguém guarda o seu endereço. Não há como liberá-la.
- Regra: para cada `new`, um `delete` correspondente.
- Boa prática: depois do `delete`, faça `p = nullptr;`.

UNIVALI

Rastreie e conte

PRÁTICA 1

```
int *p1, *p2;
p1 = new int;
*p1 = 42;
p2 = p1;
cout << *p1 << " | " << *p2 << "\n";
*p2 = 53;
cout << *p1 << " | " << *p2 << "\n";
p1 = new int;
*p1 = 88;
cout << *p1 << " | " << *p2 << "\n";
```

- Preveja as **três** saídas.
- Quantos `new` há no código?
- Quais `delete` são necessários para não vazar memória?

UNIVALI

Vetores dinâmicos: new[]

```
int n;
cin >> n;
int *v = new int[n];

for (int i = 0; i < n; i++)
    cin >> v[i];

// ... usa o vetor ...

delete[] v;
```

- `new int[n]` reserva `n` inteiros consecutivos e devolve o endereço do primeiro.
- Depois, use `v[i]` como em qualquer vetor.
- Libere com `delete[] v`.

UNIVALI

new[] e delete[] andam juntos

Alocação	Liberação
`p = new int;`	`delete p;`
`v = new int[n];`	`delete[] v;`

Erro comum: liberar um vetor com `delete` sem colchetes.

```
int *v = new int[5];
delete v;       // ERRADO: deveria ser delete[] v;
```


UNIVALI

Média sem desperdício

PRÁTICA 2

Escreva um programa que **leia a quantidade n de notas**, aloque um vetor de `float` com esse tamanho, leia as notas, imprima a **média** e libere a memória.

Exemplo de execução:

```
n: 3
7 8 6
Média: 7
```


UNIVALI

Quatro erros clássicos

1. Vazamento

```
int *p = new int;
p = nullptr;    // perdeu o endereço
```

2. Uso depois do delete

```
delete p;
cout << *p;     // p aponta para lixo
```

3. delete duplo

```
delete p;
delete p;       // libera duas vezes
```

4. Colchetes trocados

```
int *v = new int[3];
delete v;       // falta o []
```


UNIVALI

Qual é o erro de cada trecho?

PRÁTICA 3

Trecho A

```
int *v = new int[5];
v[0] = 1;
v = new int[10];
delete[] v;
```

Trecho B

```
int *p = new int;
*p = 7;
delete p;
cout << *p;
```

Trecho C

```
int *v = new int[3];
v[0] = 1;
delete v;
```

Identifique o erro em cada trecho e escreva a correção.

UNIVALI

Funções que devolvem vetores

Correto: vetor no heap

```
int* dobro(int a[], int n) {
    int *t = new int[n];
    for (int i = 0; i < n; i++)
        t[i] = 2 * a[i];
    return t;
}
```

Errado: vetor local

```
int* dobro(int a[], int n) {
    int t[100];
    for (int i = 0; i < n; i++)
        t[i] = 2 * a[i];
    return t;    // t morre aqui
}
```

- O vetor local deixa de existir quando a função termina. O ponteiro devolvido aponta para lixo.
- O vetor do heap sobrevive. Quem **recebe** o ponteiro deve fazer o `delete[]`.

UNIVALI

Quem libera?

PRÁTICA 4

```
int* dobro(int a[], int tam) {
    int *temp = new int[tam];
    for (int i = 0; i < tam; i++)
        temp[i] = 2 * a[i];
    return temp;
}

int main() {
    int a[] = {1, 2, 3, 4, 5};
    int *b = dobro(a, 5);
    for (int i = 0; i < 5; i++)
        cout << b[i] << " ";
    delete[] b;
}
```

- O que o programa imprime?
- Quem libera o vetor `b`, e onde?
- O que aconteceria se `temp` fosse um vetor local?

UNIVALI

Vetor que cresce

```
int cap = 2, n = 0;
int *v = new int[cap];
int x;
cin >> x;
while (x != 0) {
    if (n == cap) {     // cheio
        int *novo = new int[cap * 2];
        for (int i = 0; i < n; i++)
            novo[i] = v[i];
        delete[] v;
        v = novo;
        cap = cap * 2;
    }
    v[n++] = x;
    cin >> x;
}
```

- Quando o vetor enche: crie um **maior**, **copie** os dados, **libere** o antigo e passe a usar o novo.
- A capacidade dobra a cada vez.

UNIVALI

Lendo até o zero

PRÁTICA 5

Transforme o trecho de crescimento do slide anterior em uma função e use-a em um programa que **lê números até o zero** e, ao final, imprime **quantos foram lidos e a soma**.

```
int* cresce(int *v, int n, int novaCap);
```

- Dica: a função recebe o vetor antigo e devolve o novo.

UNIVALI

Vetor dinâmico de structs

```
struct Aluno {
    string nome;
    float nota;
};

int n;
cin >> n;
Aluno *turma = new Aluno[n];
for (int i = 0; i < n; i++)
    cin >> turma[i].nome >> turma[i].nota;

delete[] turma;
```

- Cada elemento é uma struct: `turma[i].nota`.
- Equivale a `(turma + i)->nota`.
- Mesma regra: `new[]` com `delete[]`.

UNIVALI

O tamanho vem do arquivo

turma.txt:

```
3
Ana 7.5
Bruno 5
Carla 9
```

```
ifstream arq("turma.txt");
if (!arq.is_open()) return 1;

int n;
arq >> n;
Aluno *turma = new Aluno[n];
for (int i = 0; i < n; i++)
    arq >> turma[i].nome >> turma[i].nota;
arq.close();

// ... usa os dados ...
delete[] turma;
```

- Sem estimar tamanho: aloca exatamente o que o arquivo informa.

UNIVALI

Turma do arquivo

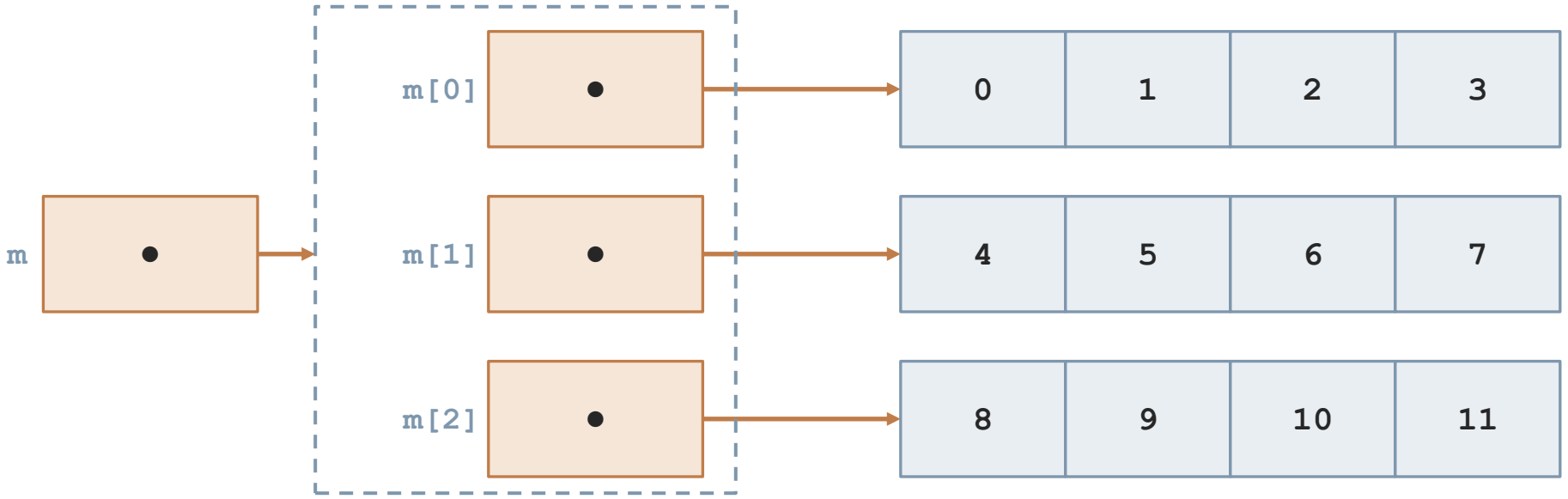
PRÁTICA 6

Usando o arquivo `turma.txt` do slide anterior, escreva um programa que aloque o vetor de `Aluno`, calcule e imprima a **média da turma** e o **nome dos aprovados** (nota >= 6). Libere a memória ao final.

- Trate o caso de arquivo inexistente.

UNIVALI

Matrizes dinâmicas: um vetor de vetores



- `m` é um `int **`: ponteiro para um vetor de ponteiros.
- Cada `m[i]` aponta para uma linha de inteiros. `m[i][j]` funciona como em qualquer matriz.

UNIVALI

Forma A: linha por linha

```
int **m = new int*[L];        // L ponteiros
for (int i = 0; i < L; i++)
    m[i] = new int[C];      // cada linha com C inteiros

m[1][2] = 7;                  // uso normal

for (int i = 0; i < L; i++)
    delete[] m[i];            // libera cada linha
delete[] m;                   // libera o vetor de ponteiros
```

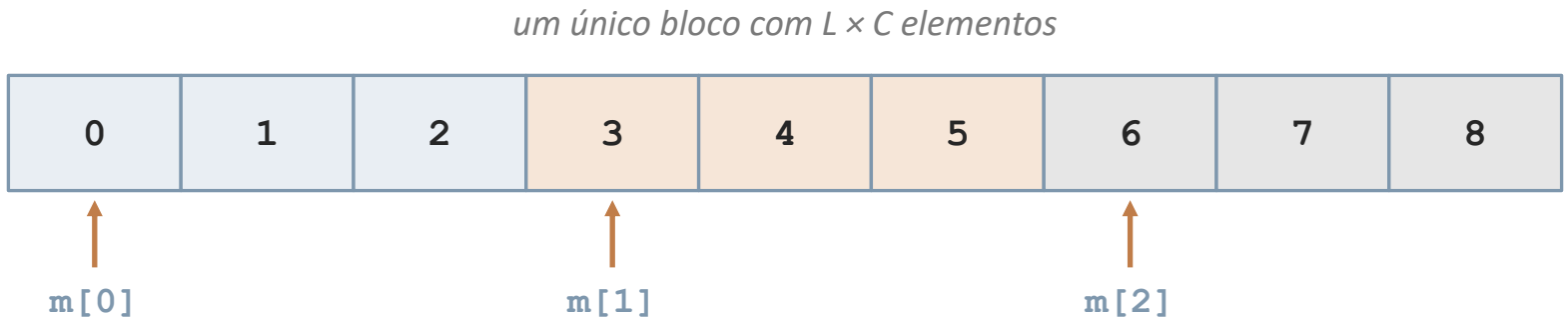
- Alocação em **dois níveis**: primeiro o vetor de ponteiros, depois cada linha.
- A liberação segue a ordem **inversa**: linhas primeiro, vetor de ponteiros por último.

UNIVALI

Forma B: um bloco só

```
int **m = new int*[L];
m[0] = new int[L * C];          // um bloco com tudo
for (int i = 1; i < L; i++)
    m[i] = m[i - 1] + C;        // cada linha começa C depois

delete[] m[0];                  // libera o bloco
delete[] m;                     // libera o vetor de ponteiros
```



UNIVALI

A ou B?

Aspecto	A: linha por linha	B: bloco único
Quantidade de `new`	L + 1	2
Quantidade de `delete[]`	L + 1	2
Elementos contíguos	não	sim
Linhas de tamanhos diferentes	sim	não

- A é mais flexível e mais comum em exercícios.
- B é mais eficiente e facilita copiar ou gravar a matriz inteira.
- Em ambos, acessamos `m[i][j]` da mesma forma.

UNIVALI

Matriz sob medida

PRÁTICA 7

Escreva um programa que **leia L e C**, aloque uma matriz de inteiros (forma A ou B), preencha com `m[i][j] = i + j`, imprima e **libere** a memória.

Exemplo (L = 3, C = 4):

```
0 1 2 3
1 2 3 4
2 3 4 5
```


UNIVALI

Matrizes e funções

```
int** criaMatriz(int l, int c) {
    int **m = new int*[l];
    for (int i = 0; i < l; i++)
        m[i] = new int[c];
    return m;
}

void liberaMatriz(int **m, int l) {
    for (int i = 0; i < l; i++)
        delete[] m[i];
    delete[] m;
}
```

- `criaMatriz` devolve um `int **` pronto para usar.
- `liberaMatriz` precisa do número de linhas `l`.
- Para percorrer, passe também `l` e `c`.

UNIVALI

Transposta

PRÁTICA 8

Usando `criaMatriz` e `liberaMatriz`, escreva a função **transposta**, que devolve uma **nova** matriz de `c` linhas e `l` colunas.

```
int** transposta(int **m, int l, int c);
```

Exemplo (2 × 3 → 3 × 2):

```
1 2 3          1 4
4 5 6    →     2 5
               3 6
```

- No `main`, libere as **duas** matrizes.

UNIVALI

Resumo

Para	Aloque com	Libere com
uma variável	`new int`	`delete p`
um vetor	`new int[n]`	`delete[] v`
uma matriz	`new int*[L]` e `new int[C]` por linha	`delete[] m[i]` e `delete[] m`

- Para cada `new`, um `delete` (e `new[]` com `delete[]`).
- Nunca use a memória depois de liberá-la.
- Quem recebe um ponteiro dinâmico precisa saber **quem é responsável por liberar**.

UNIVALI

E no C++ moderno?

```
vector<int> v(n);    // aloca e libera sozinho
v.push_back(7);      // cresce quando necessário
```

- Na prática profissional, **`vector`** faz o **`new`**, o **`delete[]`** e o crescimento por você.
- Entender alocação dinâmica explica **como** ele funciona e por que ele existe.
- Aqui, usamos **`new`** e **`delete`** para aprender o mecanismo.

UNIVALI